Unit II

Tuple

- Just like list, tuple is also an ordered collection of Python objects.
- The only difference between tuple and list is that **tuples are immutable**
- **i.e. tuple cannot be modified after it is created.**
- It is represented by tuple class.

- **Creating Tuple**
- In Python, tuples are created by placing a sequence of values separated by ‘comma’ with or without the use of parentheses for grouping of the data sequence.
- Tuples can also be created using *tuple()*
- Tuples can contain any number of elements and of any datatype (like strings, integers, list, etc.).
- **Accessing elements of Tuple**
- In order to access the tuple items refer to the index number.
- Use the index operator [ ] to access an item in a tuple.
- The index must be an integer. Nested tuples are accessed using nested indexing

Creating and accessing elements of a Tuple

- *# Creating an empty tuple*
- Tuple1 = ()
- print("Initial empty Tuple: ")
- print (Tuple1)
-
- *# Creating a Tuple with the use of Strings*
- Tuple1 = (‘Work', 'For')
- print("\nTuple with the use of String: ")
- print(Tuple1)

- *# Creating a Tuple with the use of built-in function*
- Tuple1 = tuple(‘Whoops')
- print("\nTuple with the use of function: ")
- print(Tuple1)
-

- *# Creating a Tuple with# the use of list*
- list1 = [1, 2, 4, 5, 6]
- print("\nTuple using List: ")
- print(tuple(list1))

- *# Creating a Tuple # with nested tuples*
- Tuple1 = (0, 1, 2, 3)
- Tuple2 = ('python', ‘Nerd')
- Tuple3 = (Tuple1, Tuple2)
- print("\nTuple with nested tuples: ")
- print(Tuple3)
- print("First element of tuple")
- print(Tuple1[0])
- print(“Last element of tuple")
- print(Tuple1[-1])

Set

- In Python, Set is an **unordered collection** of data type that is **iterable, mutable and has no duplicate elements.**
- The order of elements in a set is undefined though it may consist of various elements.
- **Basic uses include**
  - membership testing and
  - eliminating duplicate entries. How??
- Set objects also support mathematical operations like union, intersection, difference, and symmetric difference.
- **Creating Sets:**
- Curly braces {} or the set() function can be used to create sets.
- **Note:** to create an empty set you have to use set(), not {};
- {} creates an empty dictionary,
- Type of elements in a set need not be the same, various mixed-up data type values can also be passed to the set.

What to use when

- Lists are slightly faster than sets when you just want to **iterate over the values.**
- Sets, however, are significantly faster than lists if you want to perform **search operation**. They can only **contain unique** items though.
- Tuples perform in almost exactly the same way as lists, except for their immutability.

Set Example

- **basket = {'apple', 'orange', 'apple', 'pear', 'orange', 'banana'}**
- print(basket) # show that duplicates have been removed
- {'orange', 'banana', 'pear', 'apple'}

- **>>> 'orange' in basket # fast membership testing**
- True
- **>>> 'crabgrass' in basket**
- False

Set operations on unique letters from two words

- a = set('abracadabra')
- b = set('alacazam')
- **a** # unique letters in a
- {'a', 'r', 'b', 'c', 'd’}

  a - b # letters in a but not in b
- o/p-> {'r', 'd', 'b’}

  a | b # letters in a or b or both
- o/p-> {'a', 'c', 'r', 'd', 'b', 'm', 'z', 'l'}
- **a & b** # letters in both a and b(common letters)
- o/p-> {'a', 'c'}
- **a ^ b** # letters in a or b but not both
- o/p-> {'r', 'd', 'b', 'm', 'z', 'l'}

Set

- The major advantage of using a set, as opposed to a list, is that it has a highly **optimized method for searching**
- The main characteristics of set are –
- Sets are an **unordered collection** of elements
- Do not have duplicate elements
- i.e. ,the order in which the elements are added into the set is not fixed, it can change frequently.
- **It is defined with curly braces{}**
- **Sets are mutable, however, only immutable objects can be stored in it.**
- **Sets** cannot be indexed by numbers

Dictionary

- Dictionary in Python is an **unordered collection** of data values
- They are used to store data values like a map, which unlike other Data Types that hold only single value
- Dictionary holds **key:value** pair.
- Key-value is provided in the dictionary to make it more optimized.
- Each key-value pair in a Dictionary is separated by a colon :
- whereas each key is separated by a ‘comma’.

Creating Dictionary

- In Python, a Dictionary can be created by placing a sequence of elements within curly {} braces, separated by ‘comma’.
- **Values** in a dictionary can be of **any datatype** and can be duplicated
- **keys** can’t be repeated and **must be immutable**.
- Dictionary can also be created by the built-in function **dict()**.
- An **empty dictionary** can be created by just placing it to curly braces{}.
- Strings are the most common type of key
- You can use many other Python objects as keys, including numbers and tuples.
- Some objects, including **lists, and dictionaries, cannot** be used as **keys** because their contents can change

- **Note – Dictionary keys are case sensitive**

Dictionary

- The main operations on a dictionary are
  - **storing a value** with some key (inserting)and
  - **extracting the value** given the key.
- It is also possible to delete a key:value pair with del.
- If you store using a key that is already in use, **the old value associated with that key is forgotten/overwritten.**
- It is an error to extract a value using a non-existent key.

- Performing **list(d)** on a dictionary returns a list of all the keys used in the dictionary, in **insertion order**
- if you want it sorted, just use sorted(d) instead.
- To **check/find** whether a key is present in the dictionary, use the **in** keyword.

Dictionary Example

- telph = {'jack': 4098, ‘john': 4139}
- **telph[‘Rose'] = 4127 # inserting a value**
- telph
- {'jack': 4098, ‘john': 4139, ‘Rose': 4127}
- telph['jack’]
- **4098 #extracting a value**
- del telph[‘john’] **# deleting a value**
- **telph[‘tim'] = 4127 #insert duplicate value**
- telph
- {'jack': 4098, ‘Rose': 4127, ‘tim': 4127}
- **list(telph) # list all keys**
- ['jack', ‘Rose', ‘tim’]
- **sorted(telph) # list all keys sorted**
- ['jack', ‘Rose', ‘tim']
- **‘Rose' in telph # search a value**
- True

The dict() constructor

- The dict() constructor builds dictionaries directly from sequences of key-value pairs:

- **d=dict([('john', 4139), ('rose', 4127), ('jack', 4098)])**

- When the keys are simple strings, it is sometimes easier to specify pairs using **keyword arguments**:
- d=dict(john=4139, rose=4127, jack=4098)
- d={' john ': 4139, ‘rose': 4127, 'jack': 4098}

Creating a Dictionary

- **# Creating an empty Dictionary**
- Dict = {}
- print("Empty Dictionary: ")
- print(Dict)
-
- **# Creating a Dictionary # with Integer Keys**
- Dict = {1: ‘Naman', 2: ‘Gagan', 3: ‘Pawan'}
- print("\nDictionary with the use of Integer Keys: ")
- print(Dict)
-
- **# Creating a Dictionary # with Mixed keys**
- Dict = {'Name': ‘Naman', 1: [1, 2, 3, 4]}
- print("\nDictionary with the use of Mixed Keys: ")
- print(Dict)
-
- **# Creating a Dictionary with dict() method**
- Dict = dict({1: ‘Naman', 2: ‘Gagan', 3: ‘Pawan'})
- print("\nDictionary with the use of dict(): ")
- print(Dict)
-
- **# Creating a Dictionary with each item as a Pair**
- Dict = dict([(1, ‘Naman'), (2, ‘Gagan')])
- print("\nDictionary with each item as a pair: ")
- print(Dict)

Accessing elements of Dictionary

- In order to access the items of a dictionary refer to its **key name**.
- Key can be used inside square brackets.
- There is also a method called **get*(key)*** that will also help in accessing the element from a dictionary.

Python program to demonstrate accessing a element from a Dictionary

- # Creating a Dictionary
- Dict = {‘subject’: ‘Python', 'name': ‘sss’, ‘sem’: ‘II’}
-
- # accessing a element value using key
- print("Accessing a element using key:")
- print(Dict['name'])

- # accessing a element value using get()# method
- print("Accessing a element using get:")
- print(Dict.get(‘name’))

List Comprehensions

- List comprehensions provide a shorthand way to **create lists**.
- Common applications are
  - to make new list where **each element** is the **result of some operations** applied to each member of **another sequence or iterable**, or
  - to create a **sublist** of those elements that **satisfy** a certain **condition**.

List Comprehensions

- For example, assume we want to create a list of squares, like:
- >>>
- >>> squares = []
- >>> for x in range(10):
- ... squares.append(x**2)
- ...
- >>> squares
- [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

List Comprehensions

- We can create a list of squares in a concise manner using:
-
- squares = [x**2 for x in range(10)]

- **A list comprehension consists of:**
- ***[brackets containing an] expression***
- ***followed by a for clause,***
- ***then zero or more, for or if clauses.***
- **The result will be a new list resulting from evaluating the expression in the context of the for and if clauses which follow it.**

List Comprehensions

- For example, this listcomp combines the elements of two lists if they are not equal:

- >>>
- >>> [(x, y) for x in [1,2,3] for y in [3,1,4] if x != y]
- [(1, 3), (1, 4), (2, 3), (2, 1), (2, 4), (3, 1), (3, 4)]

Examples

- vec = [-4, -2, 0, 2, 4]
- **>>> # create a new list with the values doubled**
- >>> [x*2 for x in vec]
- [-8, -4, 0, 4, 8]
- **>>> # filter the list to exclude negative numbers**
- >>> [x for x in vec if x >= 0]
- [0, 2, 4]
- **>>> # apply a function to all the elements**
- >>> [abs(x) for x in vec]
- [4, 2, 0, 2, 4]
- **>>> # call a method on each element**
- >>> freshfruit = [' banana', ' loganberry ', 'passion fruit ']
- **>>> [f.strip() for f in freshfruit]**
- ['banana', 'loganberry', 'passion fruit']

Examples

- >>> # create a list of 2-tuples like (number, square)
- >>> [(x, x**2) for x in range(6)]
- [(0, 0), (1, 1), (2, 4), (3, 9), (4, 16), (5, 25)]
- >>> # the tuple must be parenthesized, otherwise an error is raised
- >>> # flatten a list using a listcomp with two 'for'
- >>> vec = [[1,2,3], [4,5,6], [7,8,9]]
- >>> [num for elem in vec for num in elem]
- [1, 2, 3, 4, 5, 6, 7, 8, 9]

Modules in Python

- Modules are reusable pieces of code saved in a file with .py extension
- Python has many such built in modules which help programmers
- By default few modules come with Python installation
- There are many more modules which are not built in python but have to be installed explicitly
- you can search them at https://pypi.org/..
- Check the huge number of modules
- You can install using:
  - pip install *packagename*
- In colab:
  - ! pip install packagename

Modules and Packages

- Modules comprise of classes and methods
- Packages are collection/directory of many modules
- Library and Package are used interchangeably
- Many modules are part of the standard library and are available in your python program
- There are many ways to import a module
- Ex: you wish to import the math module. You can import it as:
- *import math*
- *from math import sqrt , where sqrt is a function in math module*
- *from math import sqrt as s*

Modules

- As your program gets longer, you may want to split it into several files for easier maintenance.
- You may also want to reuse a handy function that you've written, in other programs
- To support this, Python has a way to put definitions in a file and use them in a script
- Such a file is called a *module*
- Definitions from a module can be *imported* into other modules or into the *main* module
- A module is a file containing Python definitions and statements.
- The file name is the module name with the suffix `.py` appended.
- Within a module, the module's name (as a string) is available as the value of the global variable `__name__`
- import the module with the following command:
  import module_name

Creating Your Own Modules

- If you wish to write a number of functions in a single file and then reuse them in other programs, save the file as a module
- To save a file as a module , save it with .py extension
- To use the functions from the module in your python program, import the module in your program
- Ex: if your module file is *my_module.py*
- To import this write: *import my_module*
- and start using the functions /constants defined in the module
- For ex: if my_module has functions say add, sub, fib and you wish to import only add and fib, you can write:
  - *from my_module import add,fib*
- Now, functions can be used without module name

- *In Short:* A **module** is simply a Python file that contains functions which can be reused in other programs.

Examples

- Create a module **numutils.py** containing functions to:
- check if a number is even
- check if a number is prime
- find factorial of a number

JSON: Java Script Object Notation

- JSON is a data exchange format that is both human and machine-readable.
- JSON is a lightweight, text-based format that is independent of any programming language
- It is lightweight, requires less coding, and processes faster.
- It is a common API output in a wide variety of applications(very popular with developers)
- It helps in faster data interchange and web service results
- It also reduces the mismatch between objects in the application code and their serialized representation.
- A type of databases called **Document databases** are popular among developers because they store data in a document-model format (semi-structured) rather than relational (structured).
- Document databases offer more flexibility than relational databases because developers don't have to plan out the schemas ahead of time and they can use the same format they're using in their application code.
- **JSON is a type of a document database**
- A document database is a type of nonrelational database designed to store, retrieve, and manage document-oriented information. Rather than having a schema defined upfront, document databases allow storing data in collections consisting of documents

JSON data format

- It is a data exchange format similar to XML

JSON

```
{
        "name": "tom",
        "address": "1 green street, NY",
        "phone": "2342342345"
}
```

XML

```
<name>tom</name>
<address>1 green street, NY</address>
<phone>2342342345</phone>
```

JSON with many dictionaries?

- JSON allows **only ONE root object**.
- You cannot have multiple objects at the top level.
- So for a JSON with many dictionaries , put all in a list
- OR in another dictionary

Example using list


```
{
   "students": [
      {
         "name": "Amit",
         "roll": 101,
         "marks": 78,
         "grade": "B"
      },
      {
         "name": "Rahul",
         "roll": 105,
         "marks": 32,
         "grade": "F"
      }
   ]
}
```

Example using dictionary

```
{
    "students": {
        "101": {
            "name": "Amit",
            "marks": 78,
            "grade": "B"
        },

        "105": {
            "name": "Rahul",
            "marks": 32,
            "grade": "F"
        }    } }
```

JSON in Python: Ex

- Program 1: We will create an address book and write some records to it

- Program 2: We will read this address book

Step1 for Program 1

- Create a dictionary called **book like :**
- Add 2 records in the dictionary for say Virat
- and Dhoni



Step2 for Program 1

- Import the JSON module like
- *import JSON*

- Convert your dictionary to a string in JSON format using
- *s=JSON.dumps(book) # book:is your dictionary*
- print s and check

- Store this string to a txt file. for this open a file in write mode
- *with open("d://myfolder//file1.txt","w") as f:*
- *f.write(s)*
- This will write the string s to file1.txt
- Open the file and check its content

Steps for Program 2: Read JSON format contents from a file to Python

- Open a file in read mode
- f=open(*"d://myfolder//file1.txt","r")*
- *s= f.read() # read all contents of the file into a string*

- Print the address of Dhoni. Not possible with this String s
- So convert s to a dictionary:
- *import JSON*
- *d=JSON.loads(s)*

- Print address of Dhoni
- Also print the complete address book. how?

File Handling

- Means:
- Reading Files using Python and
- Writing to Files Using Python

File Opening Modes in python

Mode	Open File For…
r	Open an existing file for Reading
w	Open an existing file/ create a new file for Writing
a	Open an existing file/ create a ne file. Data is Appended(Adding) at the end.of file
r+	Open the file for reading and writing. The handle is positioned at the beginning of the file. Raises I/O error if the file does not exists
w+	Open the file for reading and writing. For existing file, data is truncated and over-written. The handle is positioned at the beginning of the file . The file is created if it does not exist.
a+	Open the file for reading and writing. The file is created if it does not exist. The handle is positioned at the end of the file. The data being written will be inserted at the end, after the existing data

- Files can be opened in text as well as binary modes
- Appending a `'b'` to the mode opens the file in *binary mode*.
- Binary mode data is read and written as `bytes` objects.

Opening a file to read contents:open()

- f=open(“filepath\\name”,”mode”)
- This method returns a file object
- Ex and usage:
- *with open(“c:\\myFolder\\file1.txt”,r) as f:*
- *file_contents=f.read()*
- *print(file_contents)*

- using with ensures that file is closed after reading(even if some exception occurs in between)

f.read(size)

- Reads some quantity of data and returns it as a string (in text mode) or bytes object (in binary mode).
- *size* is an optional numeric argument.
- When *size* is omitted or negative, the entire contents of the file will be read and returned
- *it's your problem if the file is twice as large as your machine's memory. Haha*
- Otherwise, at most *size* characters (in text mode) or *size* bytes (in binary mode) are read and returned.
- If the end of the file has been reached, f.read() will return an empty string ('').

Reading a file line by line

- *f=open(“filepath\\name”,”r”)*

- *for line in f:*
- *print(line)*

Read a single line at a time from a file

- f=open(“filepath\\name”,”r”)

- *f.readline()*

Reading a file into a list

- *f=open("filepath\\name","r")*
- *l1=f.readlines() # l1 is a list*

- *OR*

- *l2=list(f) # l2 is a list*

Writing to a file:write(*string_value)*

- *f=open(“filepath\\name”,”w”)*
- *f.write(“Contents of File”)*

- Other types of objects have to be converted to *string* or *byte* before writing them to a file
- *v=(“Shanon”,2345)*
- *s=str(v)*
- *f.write(s)*

Open file in read and write mode

- *f=open(“filepath\\name”,”r+”)*
- f.write(“File data is so less”)

Reading a specific number of bytes from a file

- This can be done using:
- read(n): where n is a number specifying the number of bytes to be read
- readline([n]):where n is a number specifying the number of bytes to be read

Moving ahead by a few bytes in a file: seek()

- f=open(“filename”,”r”)
- f.seek(25) # takes file pointer to 25th byte
- print(f.read()) # print all contents after 25th byte

File Handling Quiz

Which of the following commands is used to open a file in append mode in Python?

- File = open("file.txt", "r")
- File = open("file.txt", "w")
- File = open("file.txt", "a")
- File = open(file.txt", "w+")

Q2.You are writing to a file using Python. The content of the file will not be saved to the file unless you _____ the file.

- open
- close
- save
- No need to do anything python will automatically save the content to file.

Q3.Which command can be used to read the remaining lines of the file in Python?

- f.read(n)
- f.read_remaining_lines()
- f.readlines()
- f.readline()

Q4.Which of the following is incorrect file opening mode in Python?

- wb+
- xr
- ab+

Brain teaser

- **Which number is not printed by this code?**

```
try:
    print(1)
    print(20 / 0)
    print(2)
except ZeroDivisionError:
    print(3)
finally:
    print(4)
```

- 2 3 1 4

Understanding __main__

- In Python, every file has a special variable: __name__

Wwhat is the value of __name__

Situation	**Value of `__name__`**
File is run directly	`"__main__"`
File is imported	File name

Case 1: if math.utils.py is run: in it,
__name__=__main__

Case 2: if main.py imports math.utils.py
now if we run main.py,
value of __name__ in main.py**=__main__**
value of __name__ in math.utils.py =
__"math.utils"

Ex: math_utils.py (module)
✓ main.py (main program)

Understanding __main__

- For Example:
- Remember the module(my_math_module) we created?
- We shall modify it so that it can run like any other Python Program
- When you import a module in another Python script(use_module.py), the import statement is executed and the imported module(in our case gets __name__ as my_math_module)
- When we execute use_module.py, value of __name__ in use_module.py is __main_
- but value of __name__ in my_math_module is my_math_module(i. e.the module name)
- And we observe that the function call statements in my_math_module also get executed
- In this case we do not want it
- We want that the function call statements in my_math_module should be executed only when my_math_module.py script is executed standalone
- **and not** when my_math_module is used as a module by another program
- To prevent this, if modules have their executable code,
- if __name__="__main__" is used in modules,
- so that the executable code in the module is executed only if the module script is run

Understanding __main__ with an example:

Question Bank (Unit I)

- Flatten the given list l3=[[2,4,5],[67,88,99],[9,1,23]] as [2,4,5,67,88,99,9,1,23]

- basket = ['apple', 'orange', 'apple', 'pear', 'orange', 'banana'].Print the unique elements of this list in sorted order.
- Write a Python program that prints all the numbers from 0 to 6 except 3 and 6
- explain a. keyword arguments b. Default arguments
- Explain how parameter passing works in Python
- Explain with example any five methods of list data type.
- How can you use a list as stack in Python
- How can you use a list as queue in Python
- Write a function that prints the quotient as well as remainder of an integer division.

Question Bank (Unit I)

What will be the output of the following operations on the string s='MyUnitTest'?

- a.s[2:] b. s[:2] c. s[:2]+s[2:] d. s[3:6] e. s[-2:] f. s[-1:-9]

Perform the following operations on the string s = 'Administrative expenses' ?

- a.change the word "Administrative" to "Selling"
- b. Check if the string s starts with "Admin"?
- c.Will you obtain the same output if you verified for "sell" instead of "Sell"
- d. Use format method to print:'The value of s is now "Selling expenses".
-

Perform the following

- a.Convert 0.33 to integer
- b.Print the remainder of 22/3
- c.Print the letter m from 'Examination'
- d.What will be the output of 'False or not True and not False'
- e.Obtain the maximum number among the values 25, 65, 890, and 15 using a built in function

Question Bank

- What are the various data types in Python
- What is meant by sequences? Explain the different sequences in Python

Question Bank

- Create a function, called "distance_from_zero", that returns the absolute value of a single argument and prints a statement "Not Possible" if the argument provided is not a number.

Perform the following operations on the list Numbers=[10, 25, 40,50]

- a.Substitute the number 10 with the number 15.
- b. Delete the number 25 from the Numbers list.
- c. Print the third-to-last element
- d. Add the list NewNumbers=[10,80,90] to Numbers.
- e. What is the position of the value 40
- f.Sort all the numbers in the "Numbers" list from the largest to the smallest

Perform different operations on the list Numbers=[10, 25, 40,50]

- g. Convert Numbers into a tuple
- h. Remove duplicates from "Numbers".
- i. Create a tuple with elements (0, 1, 2, 3)and('python', ‘Nerd')
- j. What will be the output of :
- s1=set('abracadabra')
- print(s1)

Question Bank

- What will be the output of :
- s1=set('abracadabra')
- s2={'a','e','c','d','l','m'}
- a. print(s1)
- b. print(s1-s2)
- c. print(s1|s2)
- d. print(s1&s2)
- e.print(s1^s2)
- f.s1.pop()
- g.s1.remove()

Question Bank(Unit I)

- Create a For loop that will print all the variables from a given list multiplied by 2. Let the list contain all numbers from 1 to 10. Create it with the help of the range() function.

- Find if a number is prime. Use else in a For loop.

- Explain with example:
- zip(), reversed(), sorted()

- l=['eat','sleep','repeat']. Use enumerate() to get the following output:
- 100 eat
- 101 sleep
- 102 repeat

- dict1={1:'a',2:'b',3:'c',4:'d'}.
- Print the keys and values.

Question Bank

- questions = ['name', 'quest', 'favorite color']
- answers = ['lancelot', 'the holy grail', 'blue']
- Print the values in both **questions and answers** using a single for loop. (hint:Make use of a built in function)

- Create a new list from l=[2,4,6,8] with values raised to 2 without using loops .[Hint:Use list Comprehension.]

- l2=[' Mr. Roy', ' Eliot Trevor ', 'Mickey Haller ', 'Jeff Becoz'].
- Strip values in l2 of all the preceding and trailing spaces with a singleline of code without loop. [Hint:Use list Comprehension.]

Question Bank

- Create a module my_module with two functions :
  - to find the cube of a number
  - to find quotient and remainder of a division
  - Import the module in another python program and call the functions

- Write a Python program to store the contents of a dictionary to a file(hint : convert dictionary to string using Json and write contents to file)

- Write a Python program to read the contents of a file with a dictionary. Display the value of thenthe 3rd key in the dictionary

Question Bank

- Write a program to write contents at the end of an existing file. If the file does not exist, a new file should be created and the contents should be written to the file
- Write a program to count number of words in a text file
- Write a program to count number of lines in a text file, without using loop
- Write a program to count number of lines in a text file
- Write a program to copy the contents of one text file to another

Question Bank

- Create a module my_math_module with functions sqr, absolute. Call the functions in the module itself. Ensure that if this module is imported by another python program, the executable statements in the modules should not be executed

- Combinations and variations of all the above questions may be asked